L'implémentation d'un réseau de neurone sur FPGA permet-elle d'être plus efficace énergétiquement qu'une implémentation sur CPU ?

Does implementing a neural network on an FPGA provide better energy efficiency than a CPU-based implementation?

DUJARDIN Julien

Etudiant, filière Ingénieur en Systèmes Electroniques Embarqués
Student, Embedded Electronic Systems Engineering

Résumé—Cette étude compare l'efficacité énergétique de l'inférence d'un 50 réseau de neurones (MLP) sur cibles FPGA et CPU (STM32), en 51 conservant volontairement le format float32 pour isoler l'impact de 52 l'architecture. Nos tests sur MNIST révèlent que si le FPGA offre 53 une vitesse supérieure (jusqu'à 4x) grâce au parallélisme spatial, sa 54 consommation statique élevée pénalise l'efficacité globale en usage in- 55 termittent. À l'inverse, l'optimisation logicielle sur CPU (Cortex-M7) 56 divise l'énergie par 8,6, positionnant les microcontrôleurs comme la 57 solution la plus sobre pour les petits modèles embarqués non conti- 58 nus.

This study evaluates the energy efficiency of neural network in- 61 ference (MLP) by comparing FPGA and CPU (STM32) implemen- 62 tations. By maintaining a strict float32 precision, the methodology 63 isolates the impact of hardware architecture from algorithmic quan- 64 tization. Experimental results on the MNIST dataset demonstrate 65 that while FPGAs offer superior throughput (up to 4x faster) through 66 spatial parallelism, their high static power consumption penalizes 67 efficiency during intermittent use. Conversely, software optimization 68 on the Cortex-M7 reduces energy consumption by a factor of 8.6, es- 69 tablishing low-power microcontrollers as the most efficient solution 70 for sporadic, small-scale embedded AI tasks.

Mots clefs—*Collecte de donnée, Optimisation énergétique, Analyse statistique, FPGA/CPU*
Data collection, Energy optimization, Statistical analysis, FPGA/CPU

1. Introduction

Les modèles d'intelligence artificielle sur réseaux de neurones permettent de faire apprendre à des machines des tâches qu'il n'aurait pas été possible de programmer explicitement, ou d'optimiser des algorithmes existants.

Fondamentalement, l'Intelligence Artificielle (IA), et plus spécifiquement l'apprentissage automatique (*Machine Learning*), marque une rupture avec l'informatique traditionnelle. Là où la programmation classique repose sur des règles logiques déterministes dictées par le développeur, l'IA procède par induction : elle construit son propre modèle mathématique en analysant de grandes quantités de données pour corréler des entrées (images, sons, capteurs) à des sorties (classifications, prédictions).

Cette capacité d'abstraction ouvre des champs d'usage considérables pour les systèmes embarqués, rendant les objets « intelligents » et autonomes. On retrouve ces applications dans l'industrie 4.0 pour la maintenance prédictive (analyse vibratoire), dans la vision par ordinateur (contrôle qualité, détection de présence), ou encore dans le traitement du signal pour les interfaces vocales et la santé connectée.

Toutefois, la taille et la complexité de ces modèles influent largement sur les ressources, matérielles comme énergétiques, dont le système devra disposer. Le but de cet article est de fournir un guide qui permettra d'aider à la sélection de la technologie la plus adaptée dans une solution embarquée intégrant ces algorithmes. Deux principales familles seront confrontées : les FPGAs et les CPUs (Cœurs Temps réel/Performance/DSP). L'identification d'optimisations et leur implémentation sera également prise en compte, pour déterminer une utilisation réaliste du système.

2. Cadre de l'étude

Dans le cadre des algorithmes de réseaux de neurones, on retrouve principalement deux phases : l'entraînement, et l'usage. Nous reviendrons sur la définition et le fonctionnement d'un tel algorithme dans une autre section. La partie entraînement est celle qui va déterminer les facteurs (poids et biais) à partir des données annotées. La partie inférence va appliquer ces facteurs à l'ensemble de ces données d'entrée pour définir une prédiction. Dans ce document nous nous intéresserons seulement à la partie usage. Le inférence sur la création d'un modèle sera néanmoins détaillé rapidement.

Une des problématiques majeures de l'intelligence artificielle sur cibles embarquées réside dans la différence entre la complexité mathématique des modèles (calcul matriciel intensif) et les ressources limitées des microcontrôleurs ou FPGA (énergie, mémoire, fréquence). Traditionnellement, la littérature résout ce goulot d'étranglement par la quantification, réduisant la précision des données (de `float32` vers `int8`) pour accélérer les calculs. Cependant, cette approche introduit un biais méthodologique : elle rend difficile la distinction entre le gain de performance dû à l'algorithme et celui dû à l'architecture matérielle elle-même. Cette étude se passera donc de quantification.

Table des matières

1 Introduction **1**
2 Cadre de l'étude **1**
3 Analyse de la littérature **1**
3.1 Quantification . . . 2
3.2 Utilisation de vitis HLS . . . 2
3.3 Petits modèles . . . 2
3.4 Positionnement de l'étude . . . 2
4 Méthodologie **2**
4.1 Modèle . . . 2
4.2 Choix de l'exportation des poids . . . 3
4.3 Inférence sur CPU . . . 3
4.4 FPGA . . . 3
4.5 Mesures de consommation . . . 4
5 Résultats **5**
5.1 Modèle . . . 5
5.2 CPU . . . 5
5.3 FPGA . . . 6
6 Analyse **8**
6.1 Résultats . . . 8
6.2 Méthodologie . . . 8
7 Aspects environnementaux **8**
8 Conclusion **9**
References **9**

3. Analyse de la littérature

L'étude de l'efficacité énergétique pour l'inférence de réseaux de neurones (NN) met souvent en opposition deux approches : l'ap-

proche temporelle avec des processeurs classiques (CPU/MCU) et l'approche spatiale avec des architectures reconfigurables (FPGA). Même si le consensus académique a tendance à préférer les FPGA pour leur capacité de parallélisation, en regardant de plus près la littérature récente (2020-2025), on voit que cet avantage dépend beaucoup de deux facteurs que notre méthodologie a volontairement mis de coté : la quantification des données (souvent en INT8 ou binaire) et l'optimisation bas-niveau (RTL manuel). Cette partie analyse comment le fait de garder le format `float32` change la donne au niveau de la puissance, surtout pour des modèles compacts comme celui qu'on utilise pour MNIST.

3.1. Quantification

La plupart des travaux qui montrent que les FPGA sont meilleurs énergétiquement reposent sur une réduction importante de la précision. Une comparaison récente sur l'inférence efficace [2] montre que l'implémentation matérielle de formats à virgule flottante (comme le FP32 ou le FP8) est entre 50% et 180% moins efficace en surface et en énergie que les formats entiers (INT8). Les auteurs expliquent cela par le fait que la logique pour aligner les mantisses et gérer les exposants prends beaucoup de ressources dynamiques, alors que les opérations entières se font sur des structures plus simples.De la même façon, des travaux sur l'accélération de réseaux de neurones pour la physique des particules (LHCb) [7] montrent que les FPGA (comme les cartes Alveo) dépassent les GPU en efficacité (jusqu'à 3× moins d'énergie par événement), mais ces résultats sont obtenus avec des outils comme `hls4ml` qui quantifient énormément les modèles pour qu'ils rentrent dans les DSP et les LUTs.

Ces résultat sont impressionnants mais ne s'appliquent pas directement à notre cas. Nous avons choisi de conserver le format `float32` pour être sûr d'avoir une comparaison stricte, ce qui impose au FPGA une grosse contrainte. Contrairement au microcontrôleur STM32H7, qui a une FPU (Floating Point Unit) gravée dans le silicium et optimisée ("hard-IP"), le FPGA doit émuler ces opérations en assemblant des blocs DSP et de la logique, ce qui entraine une surconsommation de ressources et de puissance

3.2. Utilisation de vitis HLS

Notre méthodologie utilise Vitis HLS pour le développement FPGA. C'est un choix fait pour la productivité mais qui rajoute un "overhead". Une analyse comparative de 2022 sur les unités de calcul [1] a chiffré cet écart : pour des designs simples, le faire en RTL manuel (Verilog/VHDL) consomme beaucoup moins de ressources (Flip-Flops et BRAMs) et a une latence plus faible que l'équivalent généré par HLS. L'outil de synthèse a tendance à créer des machines d'états conservatrices pour garantir la fermeture temporelle ("timing closure"), ce qui rajoute des cycles de latence inutiles qui pénalisent l'efficacité énergétique.Des recherches plus récentes sur l'automatisation de la HLS [6] confirment que sans une exploration experte des pragmas (directives de compilation), le code généré est souvent sous-optimal au niveau du pipeline. Cela veut dire que pour un réseau simple comme le MLP 3 couches de notre étude, l'implémentation HLS risque de ne pas utiliser toutes les ressources de calcul, laissant une partie du FPGA "au repos" mais qui consomme quand même de la puissance statique. C'est un point important car, contrairement au STM32L4 qui a des modes de veille très efficaces (quelques μA), les FPGA d'entrée de gamme gardent une consommation statique élevée (plusieurs centaines de mW) dès qu'ils sont allumés, **malgré que** la logique ne soit pas utilisée à 100% [5]. L'utilisation du matériel, en pourcent sera décrit dans la partie résultat.

3.3. Petits modèles

Un point qu'on oublie souvent dans la littérature est la différence entre le débit (throughput) et la latence unitaire, surtout pour des modèles légers comme MNIST.Une étude de référence sur les accélérateurs neuronaux (NPU) embarqués [8] a montré un phénomène de "régression de performance" pour le dataset MNIST. Alors que l'accélérateur (Ethos-U55) améliorait l'efficacité de 53× pour un gros modèle MobileNetV2, il s'est avéré moins efficace et plus lent que le CPU Cortex-M55 tout seul pour le modèle MNIST. Les auteurs disent que c'est à cause de l'overhead de configuration et de déplacement des données : pour un modèle aussi petit, le temps de "setup" de l'accélérateur est plus long que le temps de calcul brut.Cela corrobore ce qu'on pourrait observer sur le couple STM32 vs FPGA. Le STM32, avec son architecture Von Neumann unifiée, peut faire l'inférence directement depuis sa mémoire Flash/Cache avec une latence d'initialisation quasi nulle. À l'inverse, le FPGA, surtout s'il est couplé à un processeur hôte (architecture Zynq ou MicroBlaze), doit gérer des transferts de données via des bus AXI. Pour une inférence unitaire (Batch=1), la latence de communication peut dominer le temps de calcul, ce qui rend l'accélération matérielle contre-productive énergétiquement.Les données récupérées dans les benchmarks comparatifs [3] montrent que si le FPGA peut théoriquement atteindre une latence de 15 μs contre 300 μs pour un CPU sur des taches vectorielles, la consommation instantanée du FPGA (souvent > 1 Watt pour la carte) est telle que le coût énergétique total par inférence (Joules = Watts × Secondes) reste favorable au microcontrôleur si l'éxécution du modèle n'est pas continu.

3.4. Positionnement de l'étude

La littérature identifie bien une zone où les FPGA ne sont pas viables : les petits modèles en précision flottante avec une contrainte de latence unitaire. La majorité des travaux contournent ce problème par la quantification [9] ou le traitement par lots (batching) pour amortir les coûts.Notre travail se distingue en acceptant ces contraintes "défavorables" pour évaluer la performance brute des architectures. En refusant la quantification, nous forçons le matériel à montrer ses limites. Nous cherchons à vérifier si le parallélisme spatial du FPGA (même s'il est inefficace en `float`) peut **pallier à** la lenteur séquentielle du CPU, ou si, comme le suggèrent les travaux sur les "tiny models" [4], la complexité de gestion du FPGA devient un poids énergétique injustifié pour une tache aussi simple que MNIST.

4. Méthodologie

4.1. Modèle

Cet article concerne la comparaison d'une inférence d'un modèle d'IA entre une implémentation sur CPU et une implémentation sur FPGA. Il est donc important de définir un modèle de référence qui sera exécuté sur les deux cibles. Nous avons choisi d'utiliser la référence historique pour les modèles d'IA : la base de données MNIST **mnist_ref**. Cette base de données est composée de 60k images de caractères manuscrits, en noir et blanc, d'une résolution de 28×28.

La première étape consiste donc à créer le modèle. Pour cela, le module Python PyTorch a été choisi. La création d'un modèle a permis un contrôle avancé sur son fonctionnement. Une fois le modèle créé, les poids peuvent être exportés pour être lancés en inférence sur les différentes cibles. La précision du modèle est détaillée dans la partie résultats.

Ce classifieur est un Perceptron Multicouche (MLP) peu profond (*shallow MLP*), avec une couche cachée de 32 éléments. La couche cachée utilise la fonction d'activation ReLU (*Rectified Linear Unit*). Cette architecture simple, comparée à des modèles convolutifs actuels, est une référence importante pour l'inférence de réseaux de neurones. Les 784 éléments de la couche d'entrée correspondent aux pixels de l'image ($28 \times 28 = 784$). Les 10 éléments de la couche de sortie correspondent aux 10 prédictions possibles du modèle (de 0 à 9). L'élément le plus *activé* sera considéré comme la prédiction du modèle.

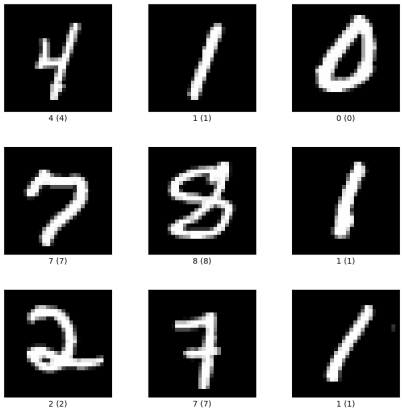

FIGURE 1. Exemple d'image issue de la base de données MNIST

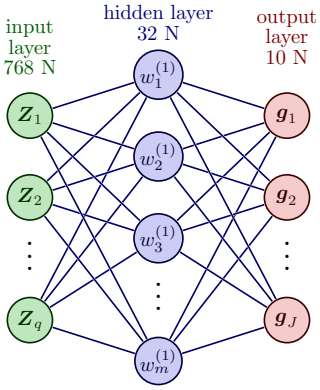

FIGURE 2. Représentation du modèle utilisé comme référence

4.2. Choix de l'exportation des poids

Le choix du format de l'exportation des poids aura un impact sur le résultat. Nous avons choisi d'utiliser le format le plus brut, c'est-à-dire le format `float`. L'impact de cette méthode, par opposition à une optimisation par quantification, est détaillé dans la partie analyse avec une comparaison des résultats si le choix avait été différent. Ce format est celui qui préserve l'intégralité de la précision des poids du modèle PyTorch, ce qui n'est pas le cas de la quantification qui introduit une perte de précision, bien que souvent négligeable.

4.3. Inférence sur CPU

4.3.1. Cibles

L'objectif de cet article concerne principalement l'embarqué. Les deux cibles embarquées choisies sont des cartes Nucleo du fabricant STMicroelectronics. Ces deux cartes offrent deux approches différentes de l'embarqué. La première, équipée d'un STM32H7 (Cortex M7), est dédiée à délivrer des performances maximales. La deuxième est équipée d'un STM32L4 (Cortex M4) dont l'intérêt est une faible consommation d'énergie.

TABLE 1. Comparaison cibles *Nucléo*

	STM32H743ZI2	STM32L476RG
Fréquence max	480 MHz	80 MHz
DMIPS	~1027	~130
FPU	Double + Single + DSP	Single
Conso typique	200 mW	20–40 mW

4.3.2. Algorithmes

Remarque

Il est important de remarquer que l'implémentation d'un même réseau de neurones, tout comme n'importe quel algorithme, peut mener à des résultats complètement différents selon la méthode et les techniques d'optimisation choisies.

Ce point peut représenter une limite pour l'article, c'est pourquoi les optimisations choisies et la méthode seront présentées dans les paragraphes suivants, tout comme une comparaison avec la méthode fournie directement par le fabricant (probablement la plus performante pour intégrer un modèle).

Ainsi, le paragraphe suivant va présenter la méthode d'implémentation et les optimisations choisies. Ces choix dépendent de plusieurs dizaines de tests ainsi que d'une recherche documentaire sur les solutions les plus courantes. L'impact des optimisations pourra être observé dans la partie résultats, en comparaison avec une implémentation sans optimisation.

Afin de maximiser l'efficacité de l'inférence sur les cibles STM32, une stratégie d'optimisation conjointe matérielle et logicielle a été adoptée. Cette approche vise à réduire les goulots d'étranglement liés aux accès mémoire et à exploiter le parallélisme d'instruction du cœur ARM Cortex. À noter que l'ensemble du code a été compilé avec le niveau d'optimisation `-Ofast`.

Activation des caches matériels L'architecture du Cortex-M7 (STM32H7) dispose de caches de niveau 1 séparés pour les instructions et les données. L'activation explicite via les instructions `SCB_EnableICache()` et `SCB_EnableDCache()` constitue la première étape critique. Cette configuration permet de stocker les segments de code fréquents (boucles de convolution) et les données (poids et activations) dans une mémoire tampon à accès immédiat. Cela réduit drastiquement les cycles d'attente (*wait-states*) imposés par les accès à la mémoire Flash ou à la RAM, particulièrement lors des calculs itératifs intenses.

Optimisations algorithmiques Le code d'inférence, initialement séquentiel, a été restructuré selon trois axes pour améliorer la localité des données et l'efficacité du pipeline :

1. **Déroulage de boucle (Loop Unrolling) :** Le produit scalaire de la couche cachée traite les entrées par blocs de quatre itérations. Cette technique diminue l'*overhead* lié à la gestion de la boucle (incrémentation et branchements conditionnels) et expose davantage d'instructions indépendantes au processeur, favorisant ainsi le parallélisme au niveau des instructions.
2. **Fusion d'opérateurs (Operator Fusion) :** Contrairement aux implémentations modulaires classiques qui calculent l'intégralité d'une multiplication matricielle avant d'appliquer le biais et l'activation, notre approche réalise une fusion verticale. L'accumulation, l'ajout du biais `b1` et la fonction ReLU sont exécutés consécutivement au sein des registres. Cette méthode élimine les écritures et lectures intermédiaires en mémoire principale, réduisant la bande passante nécessaire.
3. **Entrelacement des calculs (Software Pipelining) :** Le calcul de la couche de sortie est initié de manière anticipée, dès qu'un sous-ensemble suffisant de neurones de la couche cachée est calculé (voir logique `if (i >= 600)` dans le code). Cet entrelacement permet de réutiliser les données d'activation `hidden[i]` tant qu'elles résident encore dans les registres ou le cache de données, minimisant ainsi les défauts de cache (*cache misses*) lors de la transition entre les couches.

4.4. FPGA

4.4.1. Mise en œuvre

L'architecture choisie a été de créer un bloc (communément appelé IP dans le monde FPGA), qui embarque une image de test, le modèle

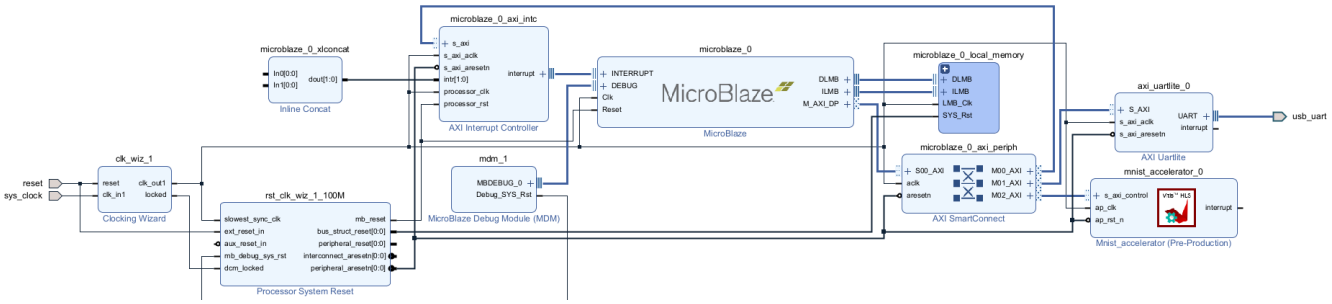

FIGURE 3. Conception du circuit sous Vivado

et ses poids, et met à disposition une interface de contrôle via le bus AXI4-Lite. Sur ce bus, on peut demander à notre bloc de faire tourner le modèle un nombre N de fois. Pour communiquer avec le modèle, nous avons choisi d'utiliser un cœur MicroBlaze dans sa forme la plus simple pour en limiter l'impact. Le but est d'avoir un système programmable avec lequel il est aisé de dialoguer afin de confirmer que le modèle fonctionne bien. Voir la figure 3 à la page 4.

Vitis HLS a été utilisé pour créer l'IP *MNIST accelerator*. L'inconvénient de cette méthode est l'agressivité de l'optimisateur intégré à Vitis. L'intégration naïve du cahier des charges ne fonctionne pas car Vitis aurait immédiatement trouvé, pendant la synthèse, que le résultat de la prédiction est toujours le même puisque l'image est embarquée. De plus, la boucle d'itération pour faire une mesure convaincante aurait été écrasée : l'IP ne se serait exécutée qu'en 1 cycle en redonnant simplement le résultat déjà calculé (là où une exécution de 25k cycles répétée 10k fois aurait dû prendre 250M de cycles).

Pour éviter ces problèmes, l'IP inclut deux procédures de protection : la première est une modification subtile à chaque itération de l'image (la prédiction reste la même, mais on s'assure qu'il n'y a pas d'optimisation possible) ; la deuxième est un accumulateur qui est également transmis en retour, ce qui empêche le compilateur d'écraser la boucle. L'impact de ces modifications sur les performances sera négligé.

4.4.2. Architecture matérielle et optimisation FPGA

Contrairement à l'approche séquentielle du CPU, l'implémentation sur FPGA repose sur une architecture spatiale où le calcul est distribué physiquement sur la puce. Le code C a été transposé en description matérielle (RTL) via un outil de synthèse de haut niveau (HLS). Cette méthode permet de définir des directives pragmatiques (`#pragma`) pour contrôler la micro-architecture générée.

L'architecture retenue conserve le format de données en virgule flottante (`float`), un choix coûteux en ressources sur FPGA mais qui permet de garder la comparaison égal avec l'implémentation sur CPU, et l'intégrité/précision du modèle.

Parallélisme d'accès mémoire (Memory Partitioning) Le goulot d'étranglement principal dans l'accélération de réseaux de neurones est souvent l'acheminement des poids vers les unités de calcul. Sur une architecture standard, la mémoire est monolithique (une lecture par cycle).

L'utilisation de directives Pragma a donc été choisi pour les poids des la première couche et l'image de référence. Dans le but de permettre un accès conccurentiel.

```
#pragma HLS ARRAY_PARTITION variable = W1 cyclic
    factor = 8 dim = 1\newcommand{\titlepos}{\
    centering}
```

Pipelining et déroulage de boucle

1. **Déroulage manuel (Unrolling) :** le programme a été concu pour permettre plusieurs niveaux de déroulage (parralélisation de process. Plus il y a de parallélisation, plus le coeur sera rapide, mais plus la consommation de matériel sera importante. Pour économiser des cycles, l'architecture du logiciel éxige que le facteur de déroulage soit un multiple de 784 (correspond aux 28×28 pixels de l'image). Ainsi nous avons 14 valeurs à notre disposition, de 1 à 392. Des essais montrent que la limite de consommation matérielle pour notre carte est de 28. Nous avons donc 1,2,4,7,8,14,16
2. **Pipelining des instructions :** La directive `#pragma HLS PIPELINE II=8` est appliquée pour cadencer le traitement. L'intervalle d'initiation (II) a été choisi pour garantir une vitesse d'exécution de 100MHz minimum.

4.5. Mesures de consommation

La mesure de la consommation a été réalisée à l'aide d'un oscilloscope et d'une résistance de 2 Ohms.

Ce qui nous intéresse, c'est la consommation d'une inférence, soit l'énergie : $E = P_{moy} \times t$, cette valeur sera en Wh (ou Joules). La puissance moyenne peut être déterminée via $P = U \times I$. U est connue pour toutes les cartes utilisées, c'est donc déterminer I qui va nous intéresser. La loi d'Ohm indique que $I = \frac{U}{R}$. Ainsi, à partir d'une mesure de tension et d'une valeur de résistance connue, nous sommes capables de déterminer I, puis P, et enfin la consommation.

Pour ce faire, nous avons utilisé une résistance de 2 Ohms en série avec l'alimentation. Avec un oscilloscope 2 voies 2204A, une mesure différentielle a été réalisée aux bornes de la résistance (mesure flottante avec masses reliées). Le logiciel Picoscope permet la création de canaux mathématiques ($\frac{A-B}{2}$) pour obtenir directement le courant.

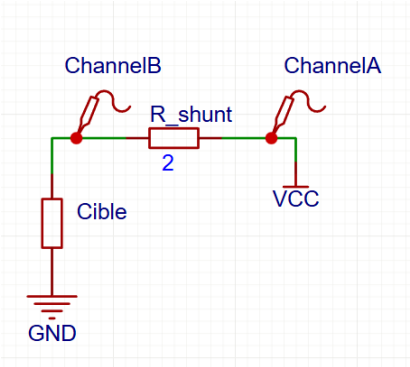

FIGURE 4. Circuit de mesure du courant

4.5.1. Points de mesure et instrumentation

Le choix des points de mesure est critique pour garantir la fidélité des résultats et minimiser le bruit induit par les composants périphériques.

Concernant les cibles **STM32 (Nucleo)**, l'architecture de la carte de développement offre un avantage significatif : la présence d'un

cavalier dédié (noté *IDD*). Ce cavalier permet d'isoler physiquement l'alimentation du microcontrôleur (MCU) du reste du circuit, notamment du programmateur ST-LINK intégré et des LEDs de statut. En insérant notre résistance shunt à la place de ce cavalier, la tension de 3.3V traverse notre dispositif de mesure avant d'alimenter exclusivement le cœur et les périphériques du STM32. Cela garantit une mesure précise de la consommation du SoC (*System on Chip*) sans le biais de l'étage de débogage.

Pour la cible **FPGA**, l'approche diffère en raison de l'absence de point de séparation simple pour le rail du cœur logique. La mesure est donc effectuée sur l'entrée d'alimentation principale de la carte (5V), en amont des régulateurs de tension. Par conséquent, la valeur mesurée englobe la consommation du FPGA, mais également celle des régulateurs, de la mémoire externe et des convertisseurs USB-UART.

4.5.2. Protocole d'interprétation des résultats

L'objectif final est d'obtenir l'énergie totale nécessaire à une inférence, exprimée en Joules ou en Watt-heures. La simple mesure de puissance instantanée ne suffit pas ; il est nécessaire de prendre en compte la durée d'exécution.

Une fois les relevés à l'ooscilloscopiquescilloscopes obtenus, nous appliquons une analyse différentielle pour séparer la consommation statique de la consommation dynamique :

1. **Mesure au repos (P_{idle}) :** Une moyenne de la puissance est calculée lorsque le processeur est initialisé mais en attente (boucle infinie ou état *idle*), sans exécuter le réseau de neurones. Cela constitue notre ligne de base ("baseline"), incluant les courants de fuite et la consommation des périphériques passifs (particulièrement critique pour la carte FPGA).
2. **Mesure en charge (P_{active}) :** Lors de l'exécution du modèle, la consommation augmente. La puissance moyenne est mesurée sur ur la fenêtre temporelle exacte de l'inférence en réalité de l'exécution multiple des inférences, entre 10k et 100k en fonction de la performance de la cible, afin de moyenner), délimitée par le basculement d'une broche GPIO (*trigger*) encadrant le début et la fin du calcul.

La puissance réellement allouée au calcul est alors estimée par $P_{net} = P_{active} - P_{idle}$. L'énergie par inférence est finalement obtenue par intégration temporelle :

$$E_{inf} = P_{net} \times \Delta t$$

Où Δt représente le temps de latence de l'inférence. Cette méthode permet de comparer équitablement le CPU (qui consomme moins mais plus longtemps) et le FPGA (qui consomme plus mais sur une durée très brève).

5. Résultats

5.1. Modèle

Le modèle développé pour cette démonstration a obtenu un score de 95.98 % de précision sur la partie *test* du jeu de données MNIST, correspondant à 10 000 images inconnues du modèle. La figure **??** correspond à la matrice de confusion, illustrant la précision du modèle pour chaque chiffre.

La génération et le test de ce modèle ont permis de créer un script dédié à la génération des fichiers de poids. Étant donné que les deux cibles utilisent le langage C (en raison de l'utilisation de la synthèse de haut niveau pour le FPGA), le format d'export privilégié est celui des fichiers d'en-tête (headers). Ainsi, 4 fichiers sont générés : `W1.h`, `W2.h`, `b1.h`, `b2.h`.

Ces fichiers contiennent un total de $25088 + 320 + 32 + 10 = 25450$ flottants pour décrire le modèle. Cela représente :

$$25450 \times 4 = 101\,800 \text{ octets} \approx 99.41 \text{ koctet}$$

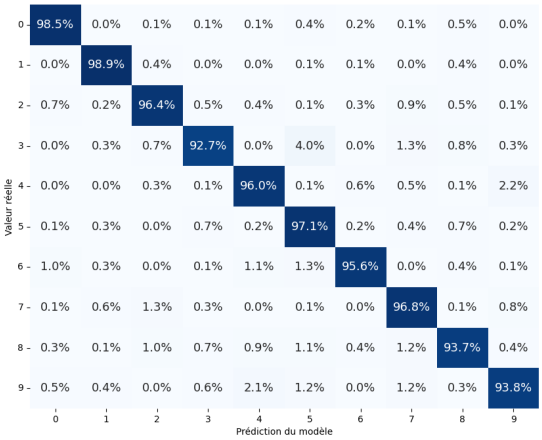

FIGURE 5. Matrice de confusion du modèle

> **Remarque**
>
> Lors de tous les essais, la précision du modèle a continué à être vérifiée. Tous les résultats de performances suivants ne traiteront plus de la question de la précision, puisque celle-ci demeure identique quel que soit le modèle ou la cible.

5.2. CPU

Comme évoqué précédemment, les choix techniques autour de l'implémentation influent directement sur le résultat final.

5.2.1. Premier essai et méthode de calcul

Les premiers essais ont été réalisés sur une implémentation simple (naïve). Les résultats initiaux ont montré un temps d'inférence de 6.58 ms. La consommation mesurée était de l'ordre de 118 mA. La consommation au repos (*idle*) pour cette carte étant d'environ 77 mA, on en déduit la consommation spécifique du modèle.

Détaillons le calcul de la puissance dynamique et de l'énergie pour 10 000 inférences. Soient I_{load} le courant en charge, I_{idle} le courant au repos et V_{dd} la tension d'alimentation (3.3 V).

Le courant consommé par l'inférence est :

$$I_{inf} = I_{load} - I_{idle} = 118 - 77 = 41 \text{ mA}$$

La puissance instantanée dissipée par le modèle est donc :

$$P_{inst} = V_{dd} \times I_{inf} = 3.3 \times 0.041 = 0.1353 \text{ W}$$

Pour quantifier l'énergie totale consommée sur 10 000 inférences, calculons d'abord le temps total d'exécution T_{total} en heures :

$$T_{total} = \frac{N_{inf} \times t_{inf}}{3600} = \frac{10^4 \times 0.00658}{3600} \approx 0.018\,28 \text{ h}$$

L'énergie totale E en milliwatt-heures (mWh) est alors :

$$E = P_{inst} \times T_{total} = 135.3 \times 0.01828 \approx 2.47 \text{ mW h}$$

Résumons cette première itération :

— Temps par inférence : 6.58 ms
— Puissance instantanée : 0.135 W
— Consommation pour 10k inférences : 2.47 mW h

Ces trois indicateurs resteront les métriques de référence peu importe la cible et l'implémentation. Ils couvrent les aspects critiques pour un concepteur de circuit embarqué : temps d'exécution, consommation instantanée et consommation énergétique totale.

5.2.2. Deuxième essai

Le deuxième essai se focalise sur l'exécution d'une implémentation optimisée, toujours sur la carte H7. L'impact individuel de chaque

optimisation ne sera pas détaillé ici.

- Temps par inférence : 0.312 ms
- Puissance consommée de l'inférence : 0.333 W
- Consommation pour 10k inférences : 0.286 mW h

Gains par rapport à la première implémentation :

- Temps par inférence : **21.1** fois plus rapide ($\frac{6.58}{0.312}$)
- Puissance instantanée : **2.46** fois plus gourmand ($\frac{0.333}{0.135}$)
- Consommation pour 10k inférences : **8.6** fois moins énergivore ($\frac{2.47}{0.286}$)

On peut en déduire que les optimisations permettent de mieux tirer profit du matériel, ce qui a un impact sur la consommation instantanée, qui est ici plus que doublée.

La figure 6 représente l'observation faîtes à l'oscilloscope. Tous les observations sont similaires on y voit généralement 4 niveaux de courant (en gris sur les graphiques). Le premier niveau de courant correspond à la consommation de la cible avant que le test commence. SOit la cible était en charge, soit elle est déjà au repos. Le courant chute, cela correspond à la commande de reset, dont le relâchement marque le début du test. Ensuite, le modèle est immédiatemment éxécuté, la cible est en charge. Puis la consommation rechute, une fois le nombre inférences terminées.

Les courbes bleus et rouges correspondent aux tensions mesurées par les sonde A et B.

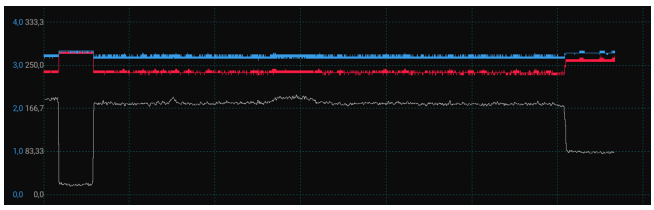

FIGURE 6. Courant consommé par le STM32H7, optimisation maximale

5.2.3. Troisième essai

L'utilisation de la bibliothèque CMSIS-DSP permet de tester l'efficacité du DSP interne de la carte. ARM met à disposition cette librairie pour exploiter au maximum cet accélérateur matériel.

- Temps par inférence : 0.437 ms
- Puissance consommée de l'inférence : 0.432 W
- Consommation pour 10k inférences : 0.525 mW h

Bien que le résultat soit intéressant par rapport au premier essai, cet accélérateur augmente la consommation instantanée et le temps par inférence par rapport à une exécution optimisée sur le CPU généraliste. Cela mène à une consommation totale presque deux fois supérieure.

5.2.4. Quatrième essai

Ce dernier essai sur cible est réalisé sur le STM32L476RG. La configuration est directement la plus optimisée. Cependant, cette puce étant moins performante que la première, certaines optimisations (comme les caches) n'ont pas pu être ajoutées en raison des limitations matérielles (espace mémoire plus faible). Le retrait de ces optimisations a un impact direct sur le résultat. La consommation au repos de la carte est environ 11 fois plus faible que la première (7 mA contre 77 mA).

- Temps par inférence : 2.769 ms
- Puissance consommée de l'inférence : 0.0346 W
- Consommation pour 10k inférences : 0.281 mW h

La consommation de l'inférence est elle aussi environ 10 fois plus faible sur cette carte (autour de 10 mA). Le temps par inférence est cependant un peu moins de 10 fois plus long, ce qui permet de réduire légèrement la consommation totale d'énergie.

Ainsi, le choix de ce microcontrôleur, plutôt qu'un modèle plus performant, peut se justifier lorsque la consommation est critique. Si le modèle ne tourne pas en continu, alors les gains de ce type de microcontrôleur sont très importants, puisqu'à mode de repos égal, la consommation est 11 fois plus faible.

5.2.5. Comparaison des résultats

STMicroelectronics met à disposition un outil en ligne pour tester le temps d'inférence d'un modèle. Toutes les optimisations possibles sont ajoutées par défaut, et la carte est dédiée à cette tâche. Des cartes physiques sont connectées aux serveurs, permettant de tester directement les performances sur cible réelle.

> **Remarque**
>
> Cet outil est cependant limité : il permet de retrouver le temps d'inférence seulement. Ni la consommation électrique, ni le résultat de la prédiction ne sont fournis. Je vais donc m'en servir uniquement comme outil de validation pour la cohérence des résultats temporels.

La carte Nucleo STMH743ZI2 est disponible sur le site.

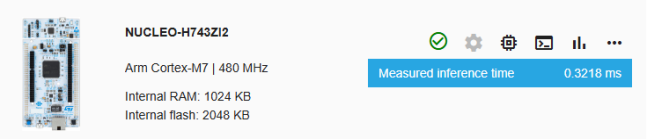

FIGURE 7. Résultat de l'outil ST pour la carte H7

On obtient une inférence de 0.3218 ms (contre 0.312 ms dans nos mesures). Cela permet de vérifier que notre temps d'inférence est cohérent, presque identique à ce qu'obtiennent les outils dédiés de ST.

Pour la carte Nucleo L476RG, elle n'est pas disponible. Cependant il existe une version proche, de la série F4. Ces dernières sont conçues pour les performances, contrairement à la série L conçue pour la basse consommation.

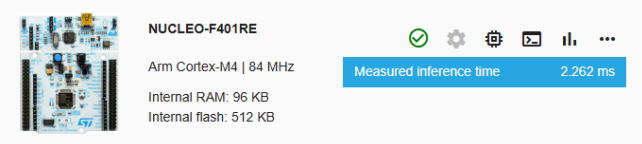

FIGURE 8. Résultat de l'outil ST pour la carte équivalente

Là encore, on obtient une inférence très proche (2.262 ms contre 2.769 ms), ce qui permet de valider la pertinence des optimisations effectuées.

5.3. FPGA

Comme indiqué dans la section méthodologie, nous allons réaliser plusieurs tests en fonction du nombre de process parallélisés. Il y aura 3 tests sur du matériel physique, et une partie simulation.

5.3.1. Premier essai

Ce premier essai concerne le nombre maximal de process admissibles sur la carte, c'est-à-dire 28.

- Temps par inférence : 0.0785 ms
- Puissance consommée de l'inférence : 0.301 W
- Consommation pour 10k inférences : 0.0656 mW h

LUT	FF	BRAM	URAM	DSP
0.00 %	**0.00 %**	**0.00 %**	**0.00 %**	**0.00 %**
71.56 %	49.10 %	80.00 %	0.00 %	62.22 %

FIGURE 9. Consommation matérielle d'une implémentation à 28 process parallèles

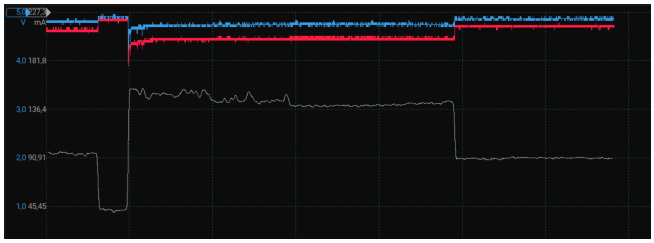

FIGURE 10. Courant consommé par le FPGA, 28 process parallélisés

5.3.2. Deuxième essai

Ce deuxième essai correspond à 2 fois moins de process parallélisés, c'est-à-dire 14.

LUT	FF	BRAM	URAM	DSP
0.00 %	**0.00 %**	**0.00 %**	**0.00 %**	**0.00 %**
55.91 %	31.60 %	80.00 %	0.00 %	33.33

FIGURE 11. Consommation matérielle d'une implémentation à 14 process parallèles

- Temps par inférence : 0.149 ms
- Puissance consommée de l'inférence : 0.285 W
- Consommation pour 10k inférences : 0.118 mW h

5.3.3. Troisième essai

Ce dernier essai se concentre sur une version avec 2 process parallèles uniquement.

LUT	FF	BRAM	URAM	DSP
0.00 %	**0.00 %**	**0.00 %**	**0.00 %**	**0.00 %**
30.93 %	16.64 %	80.00 %	0.00 %	10.00 %

FIGURE 12. Consommation matérielle d'une implémentation à 2 process parallèles

- Temps par inférence : 1.018 ms
- Puissance consommée de l'inférence : 0.011 W
- Consommation pour 10k inférences : 0.03 mW h

5.3.4. Simulations

Vitis HLS permet de connaître directement les ressources utilisées par l'IP créée, ainsi que les performances en temps d'inférence. Ces résultats (ressources et temps) sont les mêmes que ceux mesurés. La figure 13 montre le temps d'inférence en fonction du nombre de process en parallèle mais aussi la consommation matérielle associée à chaque usage.

Vivado, le logiciel utilisé pour la création de la plateforme matérielle, permet d'aller plus loin en simulant la consommation des implémentations. Les graphiques suivants sont fournis pour donner un ordre de grandeur ; la simulation se base sur tout le projet, y compris le cœur MicroBlaze. De plus, il s'agit d'une simulation statistique de l'utilisation des portes, et non la simulation de l'usage réel de l'inférence. On peut aussi noter que les valeurs réels sont prises sur l'évolution de la consommation de la carte et non le FPGA spécifiquement.

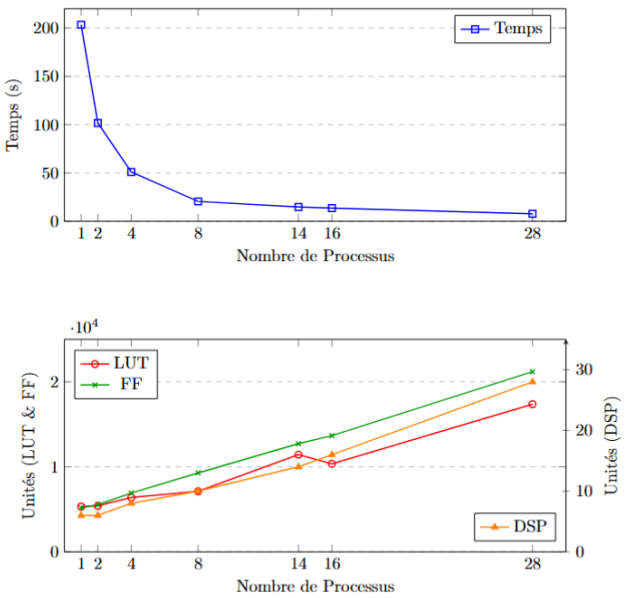

FIGURE 13. Résultats en fonction du nombre de process

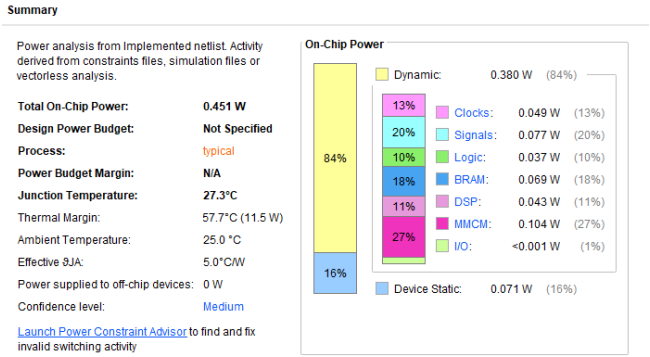

FIGURE 14. Consommation simulée pour 28 process

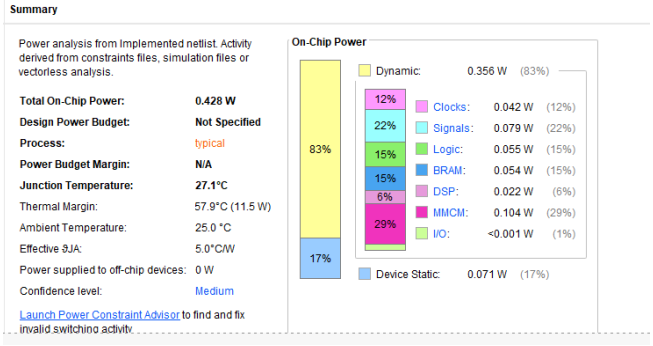

FIGURE 15. Consommation simulée pour 14 process

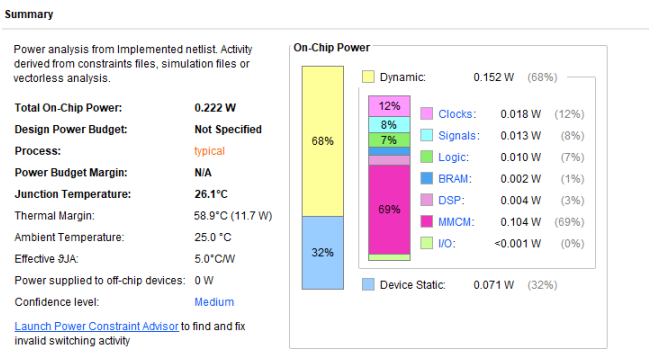

FIGURE 16. Consommation simulée pour 2 process

Comme pour les valeurs mesurées, on remarque que les deux premières implémentations ont des consommations proches. Mesures : IP28 : 0.301 W, IP14 : 0.285 W. Simulation IP28 : 0.451 W et IP14 : 0.428 W. Tandis que l'IP 2 offre une consommation bien plus faible : 0.011 W en mesure et 0.222 W en simulation.

6. Analyse

6.1. Résultats

Cet article a ainsi permis de mettre en comparaison l'implémentation d'un réseau de neurones simple sur deux cibles radicalement différentes.

Nous avons pu illustrer des tendances sur l'évolution de la consommation et des performances. Comme l'a montré la recherche bibliographique, les performances du FPGA sont bien meilleures, avec un record à 0.0785 ms contre 0.312 ms pour le CPU. Ce résultat est aussi très pertinent vis-à-vis des systèmes d'IA classiques dans les datacenters, puisque les CPUs sont réputés inefficaces, contrairement aux GPUs qui représentent la solution standard. Tout comme le FPGA, le GPU permet de paralléliser les calculs.

On remarquera cependant que le FPGA a une consommation importante même au repos. En effet, les IPs non utilisées consomment également. Par exemple, pour l'implémentation qui utilise le plus de matériel, la consommation au repos (de la carte) est de 0.450 W, contre 0.375 W pour l'implémentation minimale, au repos également.

Ainsi, dans le cas où le modèle n'est pas utilisé en continu, c'est bien un CPU, notamment *low power*, qui s'avère le plus efficace. Dans notre cas, le L476RG présente une consommation au repos (du microcontrôleur uniquement) de 7 mA seulement.

6.2. Méthodologie

Notre méthodologie peut introduire plusieurs biais.

Les résultats montrent que la cible n'est pas le seul élément dont dépendent les performances du système. La qualité de l'intégration permet aussi d'obtenir des résultats dont les variations sont importantes. Par exemple, notre STM32H7 a vu sa consommation divisée par 8.6 pour 10 000 inférences simplement en effectuant des optimisations, sans que cela ne dépende du modèle.

Les mesures de consommation du FPGA peuvent manquer de cohérence pour une confrontation directe avec les valeurs des CPUs. La raison à cela est que la méthode de calcul, qui vise à soustraire la consommation au repos de la consommation pendant une inférence, ne prend pas en compte que le FPGA a une consommation statique dépendante de l'architecture même et des IPs intégrées. Ainsi, même au repos, celles-ci consomment. Il est donc pertinent de penser que retirer la consommation au repos aura tendance à sous-estimer la consommation de l'IP pendant les inférences. Ce biais semble particulièrement vrai sur le facteur de déroulage de 2. C'est ce qu'on peut observer à la figure 17 en comparaison avec la figure 18.

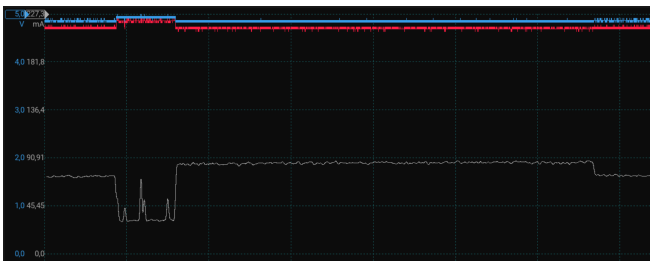

FIGURE 17. Courant consommé par le FPGA, 2 process parallélisés

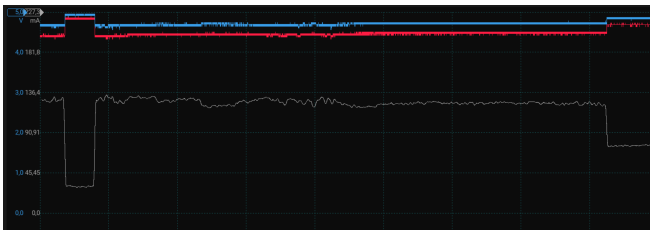

FIGURE 18. Courant consommé par le FPGA, 14 process parallélisés

Malgré ce biais, les tendances observées sont pertinentes. La différence de consommation entre les deux technologies est telle (notamment dans le cas du facteur de déroulage de 28) que même avec une sous-estimation de 10 % ou 20 %, la conclusion reste la même.

Enfin, cette étude aurait pu ajouter la comparaison avec un modèle quantifié. En effet, la quantification aurait permis de travailler sur des entiers, ce qui, dans la théorie, aurait donné un avantage au FPGA en permettant de paralléliser d'autant plus. Cependant, même dans ce cas défavorable, on obtient des performances bien meilleures ; on aurait donc vu la tendance se creuser, tout en restant la même.

7. Aspects environnementaux

En cohérence avec la Stratégie Nationale Bas Carbone (SNBC) visant la neutralité carbone, cet article peut mettre en évidence des solutions de réduction de la consommation électrique des systèmes embarqués. La phase d'usage des équipements numériques représentant une part significative de leur empreinte carbone, l'optimisation énergétique présentée ici peut répond à certains enjeux de décarbonation.

Nos résultats sur le STM32H7 démontrent que l'optimisation du code est le premier levier de réduction de l'impact environnemental, avant même le choix technologique. Le passage d'une implémentation naïve à une version optimisée a permis de diviser la consommation énergétique par un facteur **8.6** (de 2.47 mW h à 0.286 mW h pour 10 000 inférences). Cette démarche prouve qu'un travail approfondi sur l'efficacité des algorithmes permet de réduire drastiquement la demande énergétique sans nécessiter de changement matériel.

La comparaison entre le FPGA et les microcontrôleurs souligne l'importance de dimensionner la cible matérielle en fonction du profil d'usage réel pour minimiser le gaspillage énergétique :

— **Usage intensif :** Pour un flux de données continu, le FPGA offre le meilleur rendement énergétique par inférence, justifiant son coût énergétique de fabrication plus élevé par une efficacité maximale en fonctionnement.
— **Usage intermittent :** Pour des applications IoT classiques (inférences sporadiques), le STM32L476RG s'avère écologiquement plus pertinent grâce à une consommation au repos minimale (autour de 7 mA). Il évite la surconsommation statique aux architectures programmables comme les FPGA.

8. Conclusion

Cette étude démontre que l'efficacité énergétique d'une implémentation de réseau de neurones dépend moins de la technologie brute que du profil d'usage. Si le FPGA surpasse largement le CPU en performance pure et en coût énergétique par inférence grâce au parallélisme spatial (0,0656 mWh contre 0,286 mWh), sa consommation statique élevée le dessert pour des applications intermittentes. Pour des tâches légères comme MNIST en float32, un microcontrôleur optimisé (STM32L4) reste la solution la plus sobre grâce à son courant de repos minimal.

Références

[1] M. Blott et al., « On the RTL Implementation of FINN Matrix Vector Unit », *Technical Report, Xilinx Research Labs*, 2022, Étude quantitative de l'overhead HLS vs RTL manuel. adresse : https://www.scribd.com/document/652102516/FINN-RTL.

[2] A. Kuzmin, M. Nagel, M. van Baalen, A. Behboodi et T. Blankevoort, « FP8 versus INT8 for efficient deep learning inference », *arXiv preprint arXiv:2303.17951*, 2023, Analyse critique de l'efficacité hardware des formats flottants vs entiers. adresse : https://arxiv.org/abs/2303.17951.

[3] G. U. R. Group, « Deterministic Neural Network Inference: STM32 vs FPGA », *6th AccML Workshop Paper*, 2024. adresse : https://accml.dcs.gla.ac.uk/papers/2024/6th_AccML_paper_18.pdf.

[4] W. Roth et al., « Resource-Efficient Neural Networks for Embedded Systems », *Journal of Machine Learning Research*, t. 25, 2024. adresse : https://www.jmlr.org/papers/v25/18-566.html.

[5] Y. Zhang et al., « Power Consumption Benchmark for Embedded AI Inference », *ResearchGate Publication 385300510*, 2024, Comparaison conso statique/dynamique FPGA vs GPU/ASIC. adresse : https://www.researchgate.net/publication/385300510.

[6] J. Choi et al., « TimelyHLS: LLM-Driven Optimization for High-Level Synthesis », *arXiv preprint arXiv:2507.17962*, 2025, Problèmes de timing closure et d'optimisation en HLS. adresse : https://arxiv.org/abs/2507.17962.

[7] O. Djuvsland et al., « ETX4VELO: FPGA-based Neural Network Inference for LHCb Trigger », *arXiv preprint arXiv:2502.02304*, 2025, Comparaison GPU vs FPGA pour la physique des hautes énergies. adresse : https://arxiv.org/abs/2502.02304.

[8] A. S. Team, « Energy Efficiency Analysis of Cortex-M55 and Ethos-U55 », *arXiv preprint arXiv:2509.17533*, 2025, Mise en évidence de la régression de performance sur MNIST avec accélérateur. adresse : https://arxiv.org/abs/2509.17533.

[9] Y. Wang et al., « A Unified Comparison of Fine-Grained Floating Point and Integer Quantization », *arXiv preprint arXiv:2510.25602*, 2025. adresse : https://arxiv.org/abs/2510.25602.

Glossaire

AXI4-Lite Protocole de communication bus utilisé pour le contrôle/commande de blocs IP avec un faible overhead de ressources.

Batching Technique consistant à regrouper plusieurs données d'entrée pour traiter une seule inférence groupée afin d'optimiser le débit.

CMSIS-DSP Bibliothèque logicielle d'ARM optimisée pour les calculs de traitement de signal sur les cœurs Cortex-M.

Consommation dynamique Puissance dissipée liée à l'activité de commutation des transistors lors de l'exécution des calculs.

Consommation statique Puissance dissipée par les courants de fuite, présente dès que le composant est sous tension, même sans calcul.

DSP (Digital Signal Processor) Bloc matériel spécialisé dans les opérations arithmétiques intensives (multiplications-accumulations).

FPU (Floating Point Unit) Unité de calcul intégrée au processeur dédiée au traitement des nombres à virgule flottante.

HLS (High-Level Synthesis) Processus de conversion d'un code de haut niveau (C/C++) en une description matérielle (RTL/VHDL).

Inférence Phase de déploiement d'un modèle d'IA où celui-ci réalise une prédiction à partir d'une entrée réelle.

IP (Intellectual Property) Bloc logique pré-conçu et réutilisable au sein d'une architecture FPGA.

Loop Unrolling Optimisation matérielle consistant à dupliquer les unités de calcul pour traiter plusieurs itérations d'une boucle en parallèle.

MNIST Base de données normalisée de chiffres manuscrits utilisée pour l'entraînement et le test de classifieurs d'images.

Overhead Surcoût en ressources ou en temps lié à la gestion du système (bus, mémoires, protocoles) plutôt qu'au calcul brut.

Pipelining Découpage d'une opération longue en étapes plus courtes pour augmenter la fréquence de travail du circuit.

Quantification Réduction de la précision numérique des poids (ex : float32 vers int8) pour alléger la charge matérielle.